



```
In [ ]: # This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/c
# For example, here's several helpful packages to load
```

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list
the files in the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets
# saved as part of the submission. You can also write temporary files to /kaggle/temp/, but they won't be saved
after the competition.
```

```
In [12]: from collections import defaultdict
from itertools import combinations
```

```
transactions_1 = [
    ['K', 'E', 'M', 'O', 'Y'],
    ['K', 'E', 'O', 'Y'],
    ['K', 'E', 'M'],
    ['K', 'M', 'Y'],
    ['C', 'E', 'I', 'K', 'O', 'O']
]
```

```
# root
#   └─ K:4
#       └─ E:3
#           └─ M:2
#               └─ O:1
#                   └─ Y:1
#               └─ O:1
#                   └─ Y:1
#           └─ M:1
#               └─ Y:1
```

```
min_support_1 = 3
min_confidence_1 = 0.70
```

```
class FPNode:
    def __init__(self, item, parent):
        self.item = item
        self.count = 1
        self.parent = parent
        self.children = {}
        self.link = None
```

```

def build_fp_tree(transactions, min_support):
    item_counts = defaultdict(int)

    # Scan thru db and return items with less support.
    for transaction in transactions:
        for item in transaction:
            item_counts[item] += 1

    item_counts = {item: count for item, count in item_counts.items()
                    if count >= min_support}
    if not item_counts:
        return None, None

    header_table = {item: [count, None] for item, count in item_counts.items()}
    root = FPNode(None, None)

    for transaction in transactions:
        filtered = [item for item in transaction if item in item_counts]
        filtered.sort(key=lambda x: item_counts[x], reverse=True) #  $K(5) > E(4)$ 

        current_node = root
        for item in filtered:
            if item in current_node.children:
                current_node.children[item].count += 1
            else:
                new_node = FPNode(item, current_node)
                current_node.children[item] = new_node

                if header_table[item][1] is None:
                    header_table[item][1] = new_node
                else:
                    link_node = header_table[item][1]
                    while link_node.link:
                        link_node = link_node.link
                    link_node.link = new_node

            current_node = current_node.children[item]

    return root, header_table

def ascend_tree(node):
    path = []
    while node.parent and node.parent.item is not None:
        node = node.parent
        path.append(node.item)
    return path

def find_conditional_patterns(base_node): # You gather all paths that end at  $x$ 

```

```

patterns = []
while base_node:
    path = ascend_tree(base_node)
    if path:
        patterns.append((path, base_node.count))
    base_node = base_node.link
return patterns

def mine_fp_tree(header_table, min_support, prefix, frequent_itemsets): #Bottom-up
    sorted_items = sorted(header_table.items(), key=lambda x: x[1][0])

    for item, (support, node) in sorted_items:
        new_prefix = prefix.copy()
        new_prefix.add(item)
        frequent_itemsets[frozenset(new_prefix)] = support

        conditional_patterns = find_conditional_patterns(node)

        conditional_transactions = []
        for path, count in conditional_patterns:
            for _ in range(count):
                conditional_transactions.append(path)

        conditional_tree, conditional_header = build_fp_tree(
            conditional_transactions, min_support
        )

        if conditional_header:
            mine_fp_tree(conditional_header, min_support,
                          new_prefix, frequent_itemsets)

def fpgrowth(transactions, min_support):
    root, header_table = build_fp_tree(transactions, min_support)
    frequent_itemsets = {}
    if header_table:
        mine_fp_tree(header_table, min_support, set(), frequent_itemsets)
    return frequent_itemsets

def generate_rules(frequent_itemsets, transactions, min_confidence):
    rules = []
    total_transactions = len(transactions)

    for itemset in frequent_itemsets:
        if len(itemset) < 2:
            continue

        itemset_support = frequent_itemsets[itemset]

        for i in range(1, len(itemset)):
            for antecedent in combinations(itemset, i):

```

```

        antecedent = frozenset(antecedent)
        consequent = itemset - antecedent

        if antecedent in frequent_itemsets:
            antecedent_support = frequent_itemsets[antecedent]
            confidence = itemset_support / antecedent_support

            if confidence >= min_confidence:
                consequent_support = frequent_itemsets.get(consequent, 0)
                if consequent_support > 0:
                    lift = confidence / (consequent_support / total_transactions)
                else:
                    lift = 0

                rules.append({
                    'antecedent': set(antecedent),
                    'consequent': set(consequent),
                    'support': itemset_support,
                    'confidence': confidence,
                    'lift': lift
                })

    return rules

print("PROBLEM 1")
print("Minimum Support:", min_support_1)
print("Minimum Confidence:", min_confidence_1)

frequent_itemsets_1 = fpgrowth(transactions_1, min_support_1)

print("\nFREQUENT ITEMSETS:")
for itemset, support in sorted(frequent_itemsets_1.items(),
                               key=lambda x: (len(x[0]), x[1]),
                               reverse=True):
    print(set(itemset), ":", support)

rules_1 = generate_rules(frequent_itemsets_1,
                        transactions_1,
                        min_confidence_1)

print("\nSTRONG ASSOCIATION RULES:")
if rules_1:
    for idx, rule in enumerate(rules_1, 1):
        print("\nRule", idx)
        print(" ", rule['antecedent'], "→", rule['consequent'])
        print("  Support:", rule['support'])
        print("  Confidence:", round(rule['confidence'], 4))
        print("  Lift:", round(rule['lift'], 4))
else:
    print("No strong rules found.")

```

PROBLEM 1

Minimum Support: 3

Minimum Confidence: 0.7

FREQUENT ITEMSETS:

{'K', 'O', 'E'} : 4

{'K', 'E'} : 4

{'O', 'E'} : 4

{'K', 'O'} : 4

{'K', 'M'} : 3

{'K', 'Y'} : 3

{'K'} : 5

{'E'} : 4

{'O'} : 4

{'M'} : 3

{'Y'} : 3

STRONG ASSOCIATION RULES:

Rule 1

{'M'} $\rightarrow$ {'K'}

Support: 3

Confidence: 1.0

Lift: 1.0

Rule 2

{'Y'} $\rightarrow$ {'K'}

Support: 3

Confidence: 1.0

Lift: 1.0

Rule 3

{'K'} $\rightarrow$ {'E'}

Support: 4

Confidence: 0.8

Lift: 1.0

Rule 4

{'E'} $\rightarrow$ {'K'}

Support: 4

Confidence: 1.0

Lift: 1.0

Rule 5

{'O'} $\rightarrow$ {'E'}

Support: 4

Confidence: 1.0

Lift: 1.25

Rule 6

{'E'} $\rightarrow$ {'O'}

Support: 4

Confidence: 1.0

Lift: 1.25

Rule 7
{'K'} → {'O'}
Support: 4
Confidence: 0.8
Lift: 1.0

Rule 8
{'O'} → {'K'}
Support: 4
Confidence: 1.0
Lift: 1.0

Rule 9
{'K'} → {'O', 'E'}
Support: 4
Confidence: 0.8
Lift: 1.0

Rule 10
{'O'} → {'K', 'E'}
Support: 4
Confidence: 1.0
Lift: 1.25

Rule 11
{'E'} → {'K', 'O'}
Support: 4
Confidence: 1.0
Lift: 1.25

Rule 12
{'K', 'O'} → {'E'}
Support: 4
Confidence: 1.0
Lift: 1.25

Rule 13
{'K', 'E'} → {'O'}
Support: 4
Confidence: 1.0
Lift: 1.25

Rule 14
{'O', 'E'} → {'K'}
Support: 4
Confidence: 1.0
Lift: 1.0

```
In [13]: from collections import defaultdict
         from itertools import combinations

         transactions_2 = [
             ['Apple', 'Banana'],
```

```

['Cherry', 'Apple'],
['Banana', 'Cherry', 'Date'],
['Apple', 'Date', 'Fig'],
['Banana', 'Cherry'],
['Apple', 'Banana', 'Date'],
['Cherry', 'Fig'],
['Banana', 'Apple'],
['Cherry', 'Date'],
['Apple', 'Banana']
]

min_support_2 = 4
min_confidence_2 = 0.60

class FPNode:
    def __init__(self, item, parent):
        self.item = item
        self.count = 1
        self.parent = parent
        self.children = {}
        self.link = None

def build_fp_tree(transactions, min_support):
    item_counts = defaultdict(int)

    for transaction in transactions:
        for item in transaction:
            item_counts[item] += 1

    item_counts = {item: count for item, count in item_counts.items()
                    if count >= min_support}

    if not item_counts:
        return None, None

    header_table = {item: [count, None] for item, count in item_counts.items()}
    root = FPNode(None, None)

    for transaction in transactions:
        filtered = [item for item in transaction if item in item_counts]
        filtered.sort(key=lambda x: item_counts[x], reverse=True)

        current_node = root
        for item in filtered:
            if item in current_node.children:
                current_node.children[item].count += 1
            else:
                new_node = FPNode(item, current_node)
                current_node.children[item] = new_node

            if header_table[item][1] is None:

```



```

def fpgrowth(transactions, min_support):
    root, header_table = build_fp_tree(transactions, min_support)
    frequent_itemsets = {}
    if header_table:
        mine_fp_tree(header_table, min_support, set(), frequent_itemsets)
    return frequent_itemsets

def generate_rules(frequent_itemsets, transactions, min_confidence):
    rules = []
    total_transactions = len(transactions)

    for itemset in frequent_itemsets:
        if len(itemset) < 2:
            continue

        itemset_support = frequent_itemsets[itemset]

        for i in range(1, len(itemset)):
            for antecedent in combinations(itemset, i):
                antecedent = frozenset(antecedent)
                consequent = itemset - antecedent

                if antecedent in frequent_itemsets:
                    antecedent_support = frequent_itemsets[antecedent]
                    confidence = itemset_support / antecedent_support

                    if confidence >= min_confidence:
                        consequent_support = frequent_itemsets.get(consequent, 0)
                        if consequent_support > 0:
                            lift = confidence / (consequent_support / total_transactions)
                        else:
                            lift = 0

                        rules.append({
                            'antecedent': set(antecedent),
                            'consequent': set(consequent),
                            'support': itemset_support,
                            'confidence': confidence,
                            'lift': lift
                        })

    return rules

print("PROBLEM 2")
print("Minimum Support:", min_support_2)
print("Minimum Confidence:", min_confidence_2)

frequent_itemsets_2 = fpgrowth(transactions_2, min_support_2)

print("\nFREQUENT ITEMSETS:")

```

```

for itemset, support in sorted(frequent_itemsets_2.items(),
                               key=lambda x: (len(x[0]), x[1]),
                               reverse=True):
    print(set(itemset), ":", support)

rules_2 = generate_rules(frequent_itemsets_2,
                        transactions_2,
                        min_confidence_2)

print("\nSTRONG ASSOCIATION RULES:")
if rules_2:
    for idx, rule in enumerate(rules_2, 1):
        print("\nRule", idx)
        print(" ", rule['antecedent'], "→", rule['consequent'])
        print("   Support:", rule['support'])
        print("   Confidence:", round(rule['confidence'], 4))
        print("   Lift:", round(rule['lift'], 4))
else:
    print("No strong rules found.")

```

PROBLEM 2

Minimum Support: 4

Minimum Confidence: 0.6

FREQUENT ITEMSETS:

{'Apple'} : 6

{'Banana'} : 6

{'Cherry'} : 5

{'Date'} : 4

STRONG ASSOCIATION RULES:

No strong rules found.

```

In [14]: from collections import defaultdict
         from itertools import combinations

         transactions_3 = [
             ['l1', 'l2', 'l3'],
             ['l2', 'l3', 'l4'],
             ['l4', 'l5'],
             ['l1', 'l2', 'l4'],
             ['l1', 'l2', 'l3', 'l5'],
             ['l1', 'l2', 'l3', 'l4']
         ]

         min_support_3 = int(0.50 * len(transactions_3))
         min_confidence_3 = 0.60

         class FPNode:
             def __init__(self, item, parent):
                 self.item = item
                 self.count = 1
                 self.parent = parent

```

```

        self.children = {}
        self.link = None

def build_fp_tree(transactions, min_support):
    item_counts = defaultdict(int)

    for transaction in transactions:
        for item in transaction:
            item_counts[item] += 1

    item_counts = {item: count for item, count in item_counts.items()
                    if count >= min_support}

    if not item_counts:
        return None, None

    header_table = {item: [count, None] for item, count in item_counts.items()}
    root = FPNode(None, None)

    for transaction in transactions:
        filtered = [item for item in transaction if item in item_counts]
        filtered.sort(key=lambda x: item_counts[x], reverse=True)

        current_node = root
        for item in filtered:
            if item in current_node.children:
                current_node.children[item].count += 1
            else:
                new_node = FPNode(item, current_node)
                current_node.children[item] = new_node

                if header_table[item][1] is None:
                    header_table[item][1] = new_node
                else:
                    link_node = header_table[item][1]
                    while link_node.link:
                        link_node = link_node.link
                    link_node.link = new_node

            current_node = current_node.children[item]

    return root, header_table

def ascend_tree(node):
    path = []
    while node.parent and node.parent.item is not None:
        node = node.parent
        path.append(node.item)
    return path

```

```

def find_conditional_patterns(base_node):
    patterns = []
    while base_node:
        path = ascend_tree(base_node)
        if path:
            patterns.append((path, base_node.count))
        base_node = base_node.link
    return patterns

def mine_fp_tree(header_table, min_support, prefix, frequent_itemsets):
    sorted_items = sorted(header_table.items(), key=lambda x: x[1][0])

    for item, (support, node) in sorted_items:
        new_prefix = prefix.copy()
        new_prefix.add(item)
        frequent_itemsets[frozenset(new_prefix)] = support

        conditional_patterns = find_conditional_patterns(node)

        conditional_transactions = []
        for path, count in conditional_patterns:
            for _ in range(count):
                conditional_transactions.append(path)

        conditional_tree, conditional_header = build_fp_tree(
            conditional_transactions, min_support
        )

        if conditional_header:
            mine_fp_tree(conditional_header, min_support,
                          new_prefix, frequent_itemsets)

def fpgrowth(transactions, min_support):
    root, header_table = build_fp_tree(transactions, min_support)
    frequent_itemsets = {}
    if header_table:
        mine_fp_tree(header_table, min_support, set(), frequent_itemsets)
    return frequent_itemsets

def generate_association_rules(frequent_itemsets, transactions, min_confidence):
    rules = []

    for itemset, support in frequent_itemsets.items():
        if len(itemset) < 2:
            continue

        items = list(itemset)
        for i in range(1, len(items)):
            for antecedent_items in combinations(items, i):
                antecedent = frozenset(antecedent_items)

```

```

        consequent = itemset - antecedent

        if len(consequent) == 0:
            continue

        antecedent_support = frequent_itemsets.get(antecedent, 0)
        if antecedent_support > 0:
            confidence = support / antecedent_support

            if confidence >= min_confidence:
                consequent_support = frequent_itemsets.get(consequent, 0)
                lift = confidence / (consequent_support / len(transactions_3))
                rules.append({
                    'antecedent': set(antecedent),
                    'consequent': set(consequent),
                    'support': support,
                    'confidence': confidence,
                    'lift': lift
                })

    return rules

print("=" * 60)
print("PROBLEM 3: Items List Dataset")
print("=" * 60)
print(f"Total Transactions: {len(transactions_3)}")
print(f"Minimum Support: {min_support_3} (50% of {len(transactions_3)} transactions)")
print(f"Minimum Confidence: {min_confidence_3 * 100}%")
print()

frequent_itemsets_3 = fpgrowth(transactions_3, min_support_3)

print("FREQUENT ITEMSETS:")
print("-" * 60)
for itemset, support in sorted(frequent_itemsets_3.items(),
                              key=lambda x: (len(x[0]), x[1]),
                              reverse=True):
    support_percentage = (support / len(transactions_3)) * 100
    print(f"{set(itemset)}: Support = {support} ({support_percentage:.1f}%)")

print("\n" + "=" * 60)
print("STRONG ASSOCIATION RULES:")
print("-" * 60)

rules_3 = generate_association_rules(
    frequent_itemsets_3,
    transactions_3,
    min_confidence_3
)

if rules_3:
    for idx, rule in enumerate(rules_3, 1):

```

```
        print(f"\nRule {idx}:")
        print(f"    {rule['antecedent']} → {rule['consequent']}")
        print(f"    Support: {rule['support']}")
        print(f"    Confidence: {rule['confidence']:.2%}")
        print(f"    Lift: {rule['lift']:.4f}")
    else:
        print("No strong association rules found with the given confidence threshold")
```

=====

PROBLEM 3: Items List Dataset

=====

Total Transactions: 6
Minimum Support: 3 (50% of 6 transactions)
Minimum Confidence: 60.0%

FREQUENT ITEMSETS:

{'l2', 'l1', 'l3'}: Support = 3 (50.0%)
{'l2', 'l1'}: Support = 4 (66.7%)
{'l2', 'l3'}: Support = 4 (66.7%)
{'l1', 'l3'}: Support = 3 (50.0%)
{'l4', 'l2'}: Support = 3 (50.0%)
{'l2'}: Support = 5 (83.3%)
{'l1'}: Support = 4 (66.7%)
{'l3'}: Support = 4 (66.7%)
{'l4'}: Support = 4 (66.7%)

=====

STRONG ASSOCIATION RULES:

Rule 1:

{'l2'} → {'l1'}
Support: 4
Confidence: 80.00%
Lift: 1.2000

Rule 2:

{'l1'} → {'l2'}
Support: 4
Confidence: 100.00%
Lift: 1.2000

Rule 3:

{'l1'} → {'l3'}
Support: 3
Confidence: 75.00%
Lift: 1.1250

Rule 4:

{'l3'} → {'l1'}
Support: 3
Confidence: 75.00%
Lift: 1.1250

Rule 5:

{'l2'} → {'l1', 'l3'}
Support: 3
Confidence: 60.00%
Lift: 1.2000

Rule 6:

$\{'l1'\} \rightarrow \{'l2', 'l3'\}$
Support: 3
Confidence: 75.00%
Lift: 1.1250

Rule 7:
 $\{'l3'\} \rightarrow \{'l2', 'l1'\}$
Support: 3
Confidence: 75.00%
Lift: 1.1250

Rule 8:
 $\{'l2', 'l1'\} \rightarrow \{'l3'\}$
Support: 3
Confidence: 75.00%
Lift: 1.1250

Rule 9:
 $\{'l2', 'l3'\} \rightarrow \{'l1'\}$
Support: 3
Confidence: 75.00%
Lift: 1.1250

Rule 10:
 $\{'l1', 'l3'\} \rightarrow \{'l2'\}$
Support: 3
Confidence: 100.00%
Lift: 1.2000

Rule 11:
 $\{'l2'\} \rightarrow \{'l3'\}$
Support: 4
Confidence: 80.00%
Lift: 1.2000

Rule 12:
 $\{'l3'\} \rightarrow \{'l2'\}$
Support: 4
Confidence: 100.00%
Lift: 1.2000

Rule 13:
 $\{'l4'\} \rightarrow \{'l2'\}$
Support: 3
Confidence: 75.00%
Lift: 0.9000

Rule 14:
 $\{'l2'\} \rightarrow \{'l4'\}$
Support: 3
Confidence: 60.00%
Lift: 0.9000

In []: